

Competitive Programming

Section 11: Intervals And Sweep Line

Competitive Programming Course

2026-07-19

Table of contents

1	Why sweep line matters	3
2	The mental picture	4
3	Vocabulary	5
4	Recognizing sweep-line problems	6
5	Endpoint conventions and tie order	7
6	Worked example 1: peak active	8
7	Worked example 2: covered length	9
7.1	First: solve this example without a sweep line	9
7.2	Why endpoints are enough	9
7.3	Now introduce the sweep line	10
7.4	The scan in slow motion	10
7.5	Compact template	11
8	Worked example 3: closed point coverage	12
9	Active count versus active set	13
10	Grouping events	14
11	Coordinate compression versus sweep line	15
12	C++ implementation notes	16
13	Python implementation notes	17
14	Common wrong approaches	18
14.1	Iterating over every coordinate	18
14.2	Wrong tie order	18
14.3	Adding length after applying current events	18
14.4	Losing query order	18
15	Local judge workflow	19
16	Submission checklist	20
17	Exercises	21
17.1	A. Peak Active	21
17.2	B. Covered Length	21
17.3	C. Point Coverage	21

1 Why sweep line matters

Many interval problems look like they need to inspect every time or every coordinate.

That is impossible when coordinates can be as large as 10^{18} .

The sweep-line idea is:

Nothing important changes between event coordinates.
Only endpoints and query points matter.

So instead of iterating over all coordinates, we sort the important coordinates and move from left to right.

This turns problems like these into sorting plus a scan:

- maximum number of active intervals;
- total length covered by at least one interval;
- number of intervals covering each query point;
- room allocation and active resource tracking;
- counting how long coverage equals exactly k .

2 The mental picture

Imagine a vertical line moving from left to right on a number line.

```
intervals:
  [1, 5)
    [3, 7)
      [7, 9)

sweep:
  x = 1  start interval 1
  x = 3  start interval 2
  x = 5  end interval 1
  x = 7  end interval 2 and start interval 3
  x = 9  end interval 3
```

At each event, we update an active count or an active set.

3 Vocabulary

Event. A coordinate where the state may change. Starts, ends, and queries are events.

Active interval. An interval currently containing the sweep position.

Endpoint convention. The rule for whether endpoints are included:

```
[l, r) includes l but excludes r  
[l, r] includes both l and r  
(l, r) excludes both endpoints
```

Tie order. The order in which events at the same coordinate are processed. Tie order encodes endpoint conventions.

Gap accumulation. Adding the length between the previous event coordinate and the current event coordinate based on the state that was active on that whole gap.

Active set. A data structure containing currently active intervals or resources. It may be just a count, a heap of end times, or a set keyed by some value.

4 Recognizing sweep-line problems

Consider sweep line when:

- objects are intervals, segments, rectangles, bookings, logins, or time spans;
- coordinates are too large to iterate over directly;
- the answer changes only at starts, ends, and query points;
- you need a maximum active count, total covered length, or coverage counts;
- sorting events feels natural;
- a problem statement uses words like “during”, “overlap”, “contained”, “active”, “covered”, “simultaneously”, or “for each point”.

The first question should be:

`What are the events?`

The second question should be:

`How should ties at the same coordinate be ordered?`

5 Endpoint conventions and tie order

This is where many wrong answers happen.

For half-open intervals $[l, r)$, an interval ending at x is not active at x , while an interval starting at x is active at x .

If we process single events one by one for maximum active count:

`end before start`

At coordinate x , this prevents $[1, x)$ and $[x, 5)$ from being counted as overlapping.

For closed intervals $[l, r]$, both endpoints count. A query at x should see intervals that start at x and intervals that end at x .

For point coverage with closed intervals:

`start before query before end`

Tie order table:

Problem	Interval convention	Tie order at same coordinate
Peak active $[l, r)$	half-open	end, then start
Covered length $[l, r)$	half-open	group all events at same coordinate
Query coverage $[l, r]$	closed	start, query, end

6 Worked example 1: peak active

Intervals:

```
[1,5), [2,6), [4,7), [7,9)
```

Events:

coordinate	event	active after event
1	start	1
2	start	2
4	start	3
5	end	2
6	end	1
7	end	0
7	start	1
9	end	0

Maximum active count is 3.

The important tie is at 7. Because intervals are half-open, $[4,7)$ and $[7,9)$ do not overlap. The end at 7 is processed before the start at 7.

7 Worked example 2: covered length

7.1 First: solve this example without a sweep line

The intervals are:

```
[1,5), [2,6), [8,10)
```

The notation $[1,5)$ means all points from 1 up to, but not including, 5. The first two intervals overlap, so we merge them:

```
[1,5) union [2,6) = [1,6)
```

The third interval is separate. Therefore the union is:

```
[1,6) union [8,10)
```

Its length is:

```
(6 - 1) + (10 - 8) = 5 + 2 = 7
```

We want that union length. We do **not** want the sum of interval lengths: $4 + 4 + 2 = 10$ would count the overlap from 2 to 5 twice.

7.2 Why endpoints are enough

Coverage can change only when we pass an endpoint. The endpoints here are:

```
1, 2, 5, 6, 8, 10
```

They divide the line into five gaps:

```
[1,2), [2,5), [5,6), [6,8), [8,10)
```

There is no endpoint inside any one gap. So, for example, every point between 2 and 5 is covered by exactly the same intervals. We can test one point in each gap:

```
point 1.5: covered by [1,5)      -> keep [1,2)
point 3.5: covered by [1,5), [2,6) -> keep [2,5)
point 5.5: covered by [2,6)      -> keep [5,6)
point 7:   covered by nothing    -> discard [6,8)
point 9:   covered by [8,10)     -> keep [8,10)
```

The kept gaps have lengths 1, 3, 1, and 2, which add to 7. The fact that two intervals cover $[2,5)$ changes nothing: the union contains each point of that gap once.

7.3 Now introduce the sweep line

Checking one point against every interval would be too slow for many intervals. Instead, while moving left to right, store:

```
covering = how many intervals currently cover the gap immediately to our right
```

For this example:

gap	[1,2)	[2,5)	[5,6)	[6,8)	[8,10)
covering	1	2	1	0	1

For union length, only zero versus nonzero matters:

```
covering > 0 -> add this gap's length once  
covering = 0 -> add nothing
```

Each interval creates two changes:

```
at l: covering increases by 1  
at r: covering decreases by 1
```

7.4 The scan in slow motion

Start at the first endpoint. Before processing events at coordinate x , the current `covering` value describes the old gap from `previous` to x .

```
at 1:  start [1,5)          covering = 1  
  
at 2:  old gap [1,2) is covered, add 2 - 1 = 1  
       start [2,6)          covering = 2  
  
at 5:  old gap [2,5) is covered, add 5 - 2 = 3  
       end [1,5)            covering = 1  
  
at 6:  old gap [5,6) is covered, add 6 - 5 = 1  
       end [2,6)            covering = 0  
  
at 8:  old gap [6,8) is not covered, add nothing  
       start [8,10)         covering = 1  
  
at 10: old gap [8,10) is covered, add 10 - 8 = 2  
       end [8,10)           covering = 0
```

The accumulated answer is $1 + 3 + 1 + 2 = 7$.

The order is always the same: first add the old gap, then apply the changes at the new endpoint. Events at x tell us about the gap to the **right** of x , not the gap we have just crossed.

7.5 Compact template

```
create (+1) start and (-1) end events
sort them by coordinate

covering = 0
previous = first event coordinate
answer = 0

for each coordinate x, handling all events at x together:
    if covering > 0:
        answer += x - previous
    apply all changes at x to covering
    previous = x
```

Grouping all events at one coordinate is natural here. That coordinate has length zero, and every change must be applied before the next gap starts.

8 Worked example 3: closed point coverage

Intervals:

```
[1,3], [2,5], [5,5]
```

Queries:

```
1 3 4 5 6
```

Closed intervals include their endpoints, so a query at 5 sees both [2,5] and [5,5].

Use event kinds:

```
0 = start  
1 = query  
2 = end
```

At the same coordinate, sorted order is:

```
start -> query -> end
```

Trace:

```
x=1: start [1,3], query 1 -> 1  
x=2: start [2,5]  
x=3: query 3 -> 2, end [1,3]  
x=4: query 4 -> 1  
x=5: start [5,5], query 5 -> 2, end [2,5], end [5,5]  
x=6: query 6 -> 0
```

Answer:

```
1 2 1 2 0
```

9 Active count versus active set

Many beginner sweep-line problems only need an integer **active**.

```
active += 1 at start
active -= 1 at end
```

Harder problems need to know something about the active intervals:

- the earliest ending active interval;
- the interval with smallest or largest weight;
- which resource/table/room is free;
- whether an active interval contains another interval.

Then you use an active set:

priority_queue / heap	earliest or latest end
set / multiset	ordered active values
map / dictionary	compressed counts

Example: minimum rooms for meetings.

```
sort meetings by start
heap = end times of currently occupied rooms
before starting a meeting, pop rooms whose end <= start
push this meeting's end
answer = max heap size
```

That is still a sweep: the active set is a heap of end times.

10 Grouping events

For length accumulation, grouping all events at the same coordinate is usually cleaner than relying on tie order.

Template:

```
sort events by coordinate
active = 0
prev = first coordinate
answer = 0

for each coordinate x:
    if active > 0:
        answer += x - prev
    apply all deltas at x
    prev = x
```

The interval convention is encoded by the events. For $[l, r)$, add $+1$ at l and -1 at r .

11 Coordinate compression versus sweep line

Coordinate compression and sweep line are related but not identical.

Use sweep line when you can process events in sorted order once.

Use coordinate compression when you need indexed data structures over the coordinates:

- Fenwick tree;
- segment tree;
- many range updates or range queries;
- 2D sweep with an inner data structure.

For this section, a sorted event list is enough.

12 C++ implementation notes

Use `long long` for coordinates and lengths.

For simple events:

```
vector<pair<long long, int>> events;
events.push_back({l, +1});
events.push_back({r, -1});
sort(events.begin(), events.end());
```

For events with multiple kinds:

```
struct Event {
    long long x;
    int kind;
    int idx;
};
```

Sort by `(x, kind)`.

When accumulating length, do not multiply or subtract in `int`.

13 Python implementation notes

Python tuples sort lexicographically:

```
events.append((1, 0, -1))
events.append((query_x, 1, query_id))
events.append((r, 2, -1))
events.sort()
```

For large input, use:

```
data = list(map(int, sys.stdin.buffer.read().split()))
```

When answering queries, store answers in an array and write them in original order.

14 Common wrong approaches

14.1 Iterating over every coordinate

This fails when coordinates are up to 10^9 or 10^{18} . You only need event coordinates.

14.2 Wrong tie order

For half-open intervals:

`[1,2)` and `[2,3)` do not overlap

If you process starts before ends at 2, you will incorrectly count overlap.

14.3 Adding length after applying current events

The gap `[prev, x)` is controlled by the active count from after `prev`, before events at `x`. Apply the length contribution first, then update events at `x`.

14.4 Losing query order

If queries are sorted as events, their output order changes. Store each query's original index.

15 Local judge workflow

Run Section 11:

```
python3 sections/11_intervals_sweep_line/check.py
```

Run the reference solutions:

```
CP_TARGET=solution python3 sections/11_intervals_sweep_line/check.py
```

The randomized tests use brute force on small coordinates:

- for peak active, try event coordinates directly;
- for covered length, mark every integer unit in small ranges;
- for point coverage, test every query against every interval.

16 Submission checklist

- Write the interval convention: $[l,r)$, $[l,r]$, or something else.
- Define event types and tie order before coding.
- Use `long long` for coordinates and lengths in C++.
- For length, add the previous gap before applying current-coordinate events.
- For queries, store original indices.
- Test intervals that touch at endpoints.
- Test negative coordinates if allowed.
- Test duplicate intervals and duplicate query points.

17 Exercises

17.1 A. Peak Active

Practice half-open endpoint ordering and maximum active count.

17.2 B. Covered Length

Practice grouping events and accumulating covered gaps.

17.3 C. Point Coverage

Practice query events, closed intervals, and restoring original query order.